
Augmentacja danych z LLM, HITL i Active Learning

Od teorii do praktyki — kompletny proces wytwarzania modeli klasyfikacyjnych z wykorzystaniem lokalnych modeli generatywnych.

radlab.dev – Research and Development Laboratory

Tutorial · ~60 min prezentacji + sesja pytań · interaktywna sesja anotacyjna

Agenda tutorialu

1

Wstęp

Czego dotyczy tutorial i jaka korzyść dla uczestników

~2 min

2

Część teoretyczno-praktyczna

Augmentacja, labelowanie, klasyfikacja vs generowanie, HITL + AL, pętla

~25 min

★

Anotacja na żywo

Interaktywna sesja: dss2026.radlab.dev

~5 min

3

Część techniczna

Sprzęt, vendorzy, LLM Router, labelowanie i augmentacja narzędziami, trening

~22 min

4

Podsumowanie i wnioski

Ocena modelu w W&B, wnioski i kierunki dalsze

~6 min

Po co ten tutorial?

Zaprezentujemy proces kontrolowanej augmentacji danych — od podstawowych pojęć po kompletny pipeline z udziałem człowieka w pętli (HITL) i Active Learningu (AL).

- ▶ Labelowanie — człowiek vs LLM
- ▶ Augmentacja — nowe przykłady treningowe
- ▶ Klasyfikacja — modele decyzyjne
- ▶ Generowanie — LLM tworzy dane
- ▶ HITL + AL — człowiek w pętli uczenia aktywnego

♦ Cel i korzyść

Skrócenie czasu wytwarzania modeli klasyfikacyjnych oraz wsparcie w sytuacjach, gdy trudno zorganizować reprezentatywny zespół anotatorów.

W trakcie: 5-minutowa sesja anotacyjna z udziałem uczestników.

01

Część teoretyczno-praktyczna

~25 min · podstawy pojęciowe i metoda

Augmentacja danych

Tworzenie nowych przykładów treningowych na bazie istniejącego zbioru

Dobra augmentacja zapewnia, że przykłady:

- ▶ są zgodne z problemem
- ▶ zachowują poprawną etykietę
- ▶ zwiększają różnorodność zbioru
- ▶ pomagają modelowi generalizować
- ▶ uzupełniają klasy słabo reprezentowane

Metody dla tekstu:

- ▶ parafrazowanie przykładów
- ▶ zmiana stylu / długości wypowiedzi
- ▶ warianty formalne i nieformalne
- ▶ literówki, slang, język potoczny
- ▶ przykłady rzadkie i graniczne

▶ Przykład (etykieta zachowana)

„Chcę zwrócić produkt...” [zwrot] → „Jak odesłać towar?” · „Czy da się zrobić zwrot?” — wszystkie warianty pozostają w tej samej klasie.

Labelowanie danych z wykorzystaniem LLM

Przypisywanie etykiet przykładom — klasyczne (człowiek) vs wspierane LLM

Szczególnie przydatne, gdy:

- ▶ mamy dużo nieoznaczonych danych
- ▶ chcemy szybko pierwszy zbiór treningowy
- ▶ potrzebne wstępne etykiety do eksperymentów
- ▶ chcemy ograniczyć koszt ręcznej anotacji
- ▶ klasy da się jasno zdefiniować w promptach

Uwaga

Etykiety LLM to tylko propozycje. Model myli się przy ironii, niejednoznaczności, braku kontekstu i slangu.

✓ Zasada

LLM nie zastępuje człowieka — przyspiesza etykietowanie. Człowiek kontroluje jakość, poprawia błędy i rozstrzyga trudne przypadki.

Klasyfikacja vs generowanie

MODEL KLASYFIKACYJNY

Wejście	„Nie mogę zalogować się do konta”
Wyjście	klasa: problem_logowania
Dodatkowo	prawdopodobieństwa klas + thresholdy
Pytanie	Do której kategorii należy przykład?
Zalety	szybki, tani, łatwy do wdrożenia

MODEL GENERATYWNY

Wejście	prompt: „Wygeneruj alternatywy zdania”
Wyjście	nowe teksty (warianty)
Rola	pomocnicza — przygotowuje dane
Pytanie	Co można wygenerować z kontekstu?
Uwaga	nie jest modelem docelowym

LLM generatywny przygotowuje dane → model klasyfikacyjny uczy się na nich podejmować decyzje.

Active Learning (AL)

Pętla zwrotna: zamiast losowo, wybieramy najbardziej informacyjne przykłady

AL wybiera przykłady, dla których model:

- ▶ ma niską pewność predykcji
- ▶ myli się między podobnymi klasami
- ▶ daje wysokie p-stwo dla kilku klas naraz
- ▶ jest niezgodny z inną regułą / modelem
- ▶ reprezentuje rzadką / problematyczną część danych

© Przykład trudnego przypadku

„Nie dostałem produktu i chcę pieniądze z powrotem”
zwrot 0.42 · reklamacja 0.39 · status_dostawy 0.15 · nieznany 0.04

Trudny przykład — poprawne oznaczenie uczy rozróżniania klas granicznych.

Które dane warto augmentować?

Nie zawsze cały zbiór po równo — liczebność klasy to nie wszystko

- ▶ klasy o najgorszych wynikach modelu
- ▶ przykłady z pogranicza kilku klas
- ▶ przypadki ważne biznesowo (wskazane przez człowieka)
- ▶ przykłady o niskiej pewności modelu
- ▶ nietypowe warianty językowe, slang, błędy

Proces kontrolowany:

1. Trenuj prosty klasyfikator
2. Sprawdź, gdzie się myli
3. Zidentyfikuj klasy problematyczne
4. Wygeneruj przykłady (LLM)
5. Walidacja (człowiek / reguły)
6. Trenuj ponownie
7. Porównaj wyniki

Human In The Loop (HITL)

Człowiek jako świadoma część procesu uczenia i walidacji

Definiuje reguły

jakie klasy istnieją, co oznaczają, gdzie biegnie granica między nimi

Sprawdza jakość

czy przykład pasuje do etykiety, jest realistyczny, nie powiela oryginału

Rozstrzyga niejasne

spójne decyzje dla przypadków granicznych („koordynator”)

Kontroluje uczenie

metryki, confusion matrix, klasy rzadkie, jakość języka

Pętla HITL z wykorzystaniem AL

Augmentacja

Jak zwiększyć i urozmaicić zbiór?

Active Learning

Które dane są warte uwagi?

Human in the Loop

Jak utrzymać jakość i kontrolę?

Przepływ:

Zbiór danych



Trening



Analiza błędów



Wybór do
augmentacji



Generowanie (LLM)



Walidacja

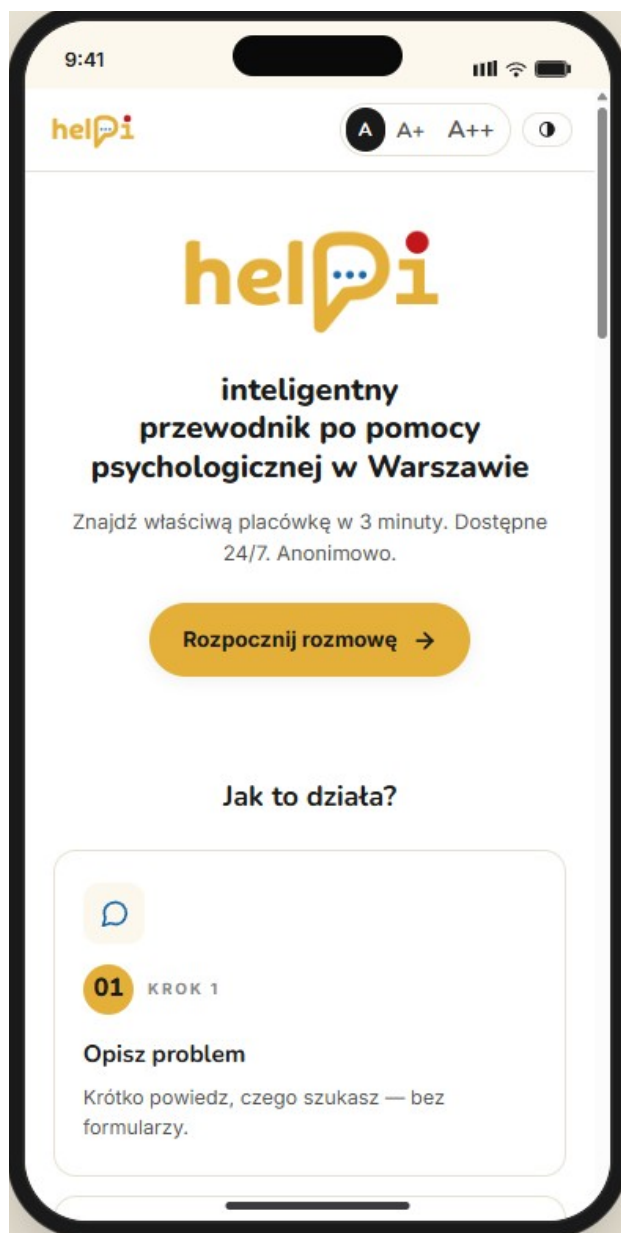


Aktualizacja



Ponowny trening

Bezpieczniejsza niż „weź dane → wygeneruj 10× więcej → trenuj”. Samo zwiększenie liczby przykładów nie gwarantuje poprawy — może nawet pogorszyć wynik.



cel: usprawnienie nawigacji beneficjentów w systemie bezpłatnej pomocy psychologicznej w Warszawie;

kryterium: przedstawienie beneficjentowi danych teleadresowych adekwatnej placówki w szybkim czasie;

produkt końcowy: Chatbot Helpi

Budowa języka i pierwszego zbioru danych

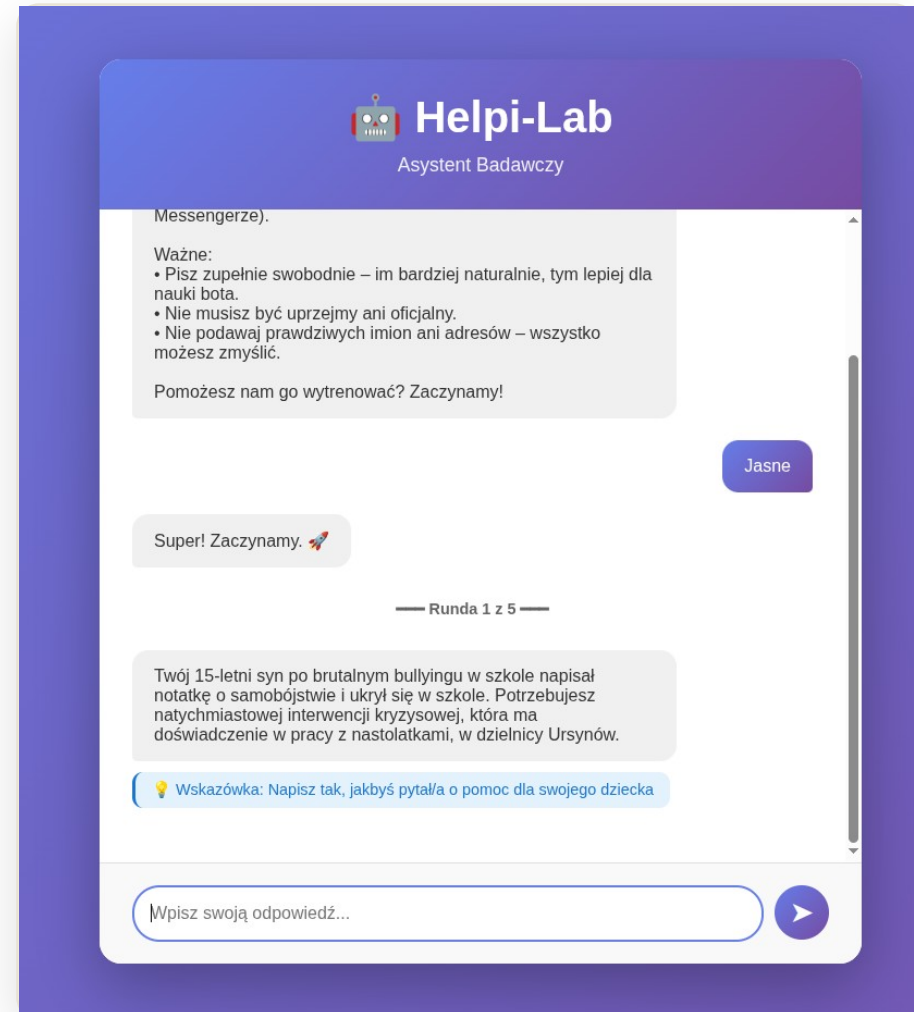
Narzędzie **Helpi-Lab** — aplikacja badawcza, w której testerzy wcielali się w różne role (rodzic szukający pomocy dla dziecka, osoba w kryzysie, student) i pisali naturalne zapytania.

680

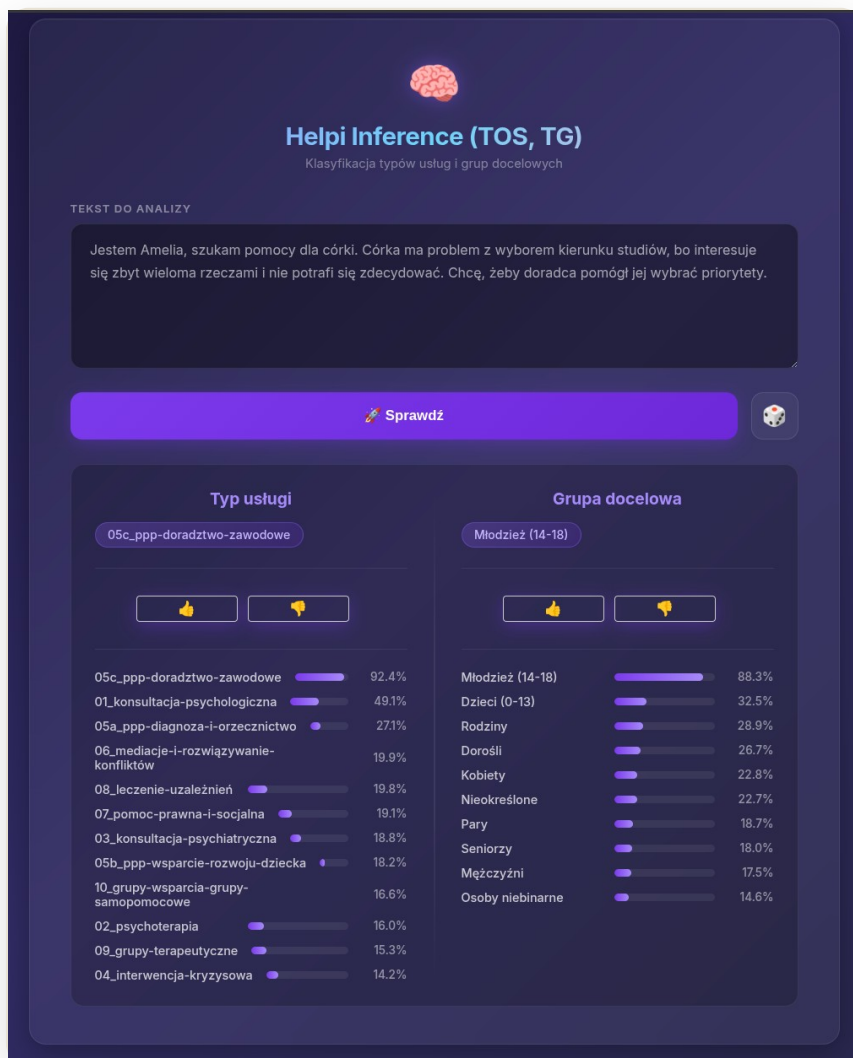
ręcznie zaanotowanych oczekiwanych zakresów tematycznych i językowych użytkownika — „seed dataset”.

TAKSONOMIA + TŁUMACZONE ZBIORY

Zbiory **mental health** z HuggingFace przetłumaczono na polski, a na podstawie przeszukiwania (crawling) stron placówek zdefiniowano: **typy usług (Types Of Services)** i **grupy docelowe (Target Groups)**.



Helpi-Lab — interfejs asystenta badawczego z rundami scenariuszy



Helpi Inference (TOS, TG) — testowanie klasyfikatorów i rozkład prawdopodobieństw

Anotacja i trening modeli klasyfikacyjnych

Model **GPT-OSS:120b** generował wstępne klasyfikacje (typ usługi TOS + grupa docelowa TG), a ludzie weryfikowali i korygowali je w aplikacji **Helpi-Lab-Chat-Anno**.

4933

sprawdzone anotacje (etykietowanie) — pierwszy „oficjalny” dataset.

2

modele klasyfikacyjne: **TOS** (typy usług) i **TG** (grupy docelowe).

Te modele stały się **rdzeniem logiki Helpi** — one decydują, jakie placówki proponować użytkownikowi na podstawie opisu problemu.

Balans - skalowanie danych i doskonalenie modeli

Wytrenowane klasyfikatory same anotowały nowo wygenerowane przypadki, a ludzie weryfikowali wyniki — pętla, która systematycznie podnosiła precyzję klasyfikacji.

PĘTLA DOSKONALENIA MODELI

1 Model trenuje



2 Anotuje nowe dane



3 Ludzie korygują

↻ **model trenuje ponownie**

~700

podtura A — przypadki typów usług

4354

podtura B — przypadki łącznie dla TOS i TG

~5400 +940

Dataset grup docelowych był niezbalansowany (nadreprezentacja „dorosłych”, niedobór „seniorów” i „osób niebinarnych”), więc dogenerowano ok. **940** przypadków — uzyskując ~**5400** zbalansowanych przykładów dla TG.

Interaktywna sesja anotacyjna

Spróbuj sam — przekonaj się, jak trudno dokładnie oznaczać tekst bez kontekstu biznesowego.

Link do sesji (~5 min):

<https://dss2026.radlab.dev/>

Przeczytaj

każdy tweet uważnie

Zastanów się

jaka jest intencja

Wybierz

najlepszą klasę

Zwróć uwagę

na niuanse i kontekst

03

Część techniczna

~22 min · od sprzętu po trening klasyfikatora

Wymagania sprzętowe

Komponent	Minimalne	Zalecane	Uwagi
CPU	4 rdzenie	8+ rdzeni	batch wielowątkowy: więcej rdzeni = szybciej
RAM	16 GB	32–128 GB+	7B: 16GB · 70B: 64GB+ · 120B: 96GB+
Dysk	50 GB	250 GB+	Bielik ~5GB · PLLuM ~40GB · gpt-oss ~65GB
GPU	opcjonalne	24–80 GB VRAM	Bielik 8GB · PLLuM 48GB+ · gpt-oss 80GB
System	Linux / macOS 13+	Ubuntu 22.04+	Linux docelowy; vLLM tylko Linux+NVIDIA

Wymagania zależą od modelu i vendora. Bez GPU używaj Ollama / llama.cpp (CPU-only). Python 3.10+ (zalecane 3.12+), izolowany venv, zależności z requirements.txt.

Lokalne silniki (vendorzy)

Trzy sposoby serwowania modeli lokalnie

vLLM	wysokowydajny, GPU NVIDIA, tensor parallelism, multi-GPU
Ollama	najprostszy, GGUF, NVIDIA/AMD/Apple/CPU, API :11434
llama.cpp	lekki C/C++, GGUF, największa kontrola, najwięcej konfiguracji

Cecha	vLLM	Ollama	llama.cpp
GPU	NVIDIA (CUDA)	ROCm/Metal/CUDA	CUDA/Metal/Vulkan
CPU-only / macOS / Win	X / X / X	✓ / ✓ / WSL2	✓ / ✓ / ✓
Łatwość konfiguracji	średnia	łatwa	trudna
Wydajność GPU / CPU	b. wysoka / brak	wysoka / dobra	wysoka / b. dobra
Modele 120B+	✓ multi-GPU	GGUF/RAM	GGUF/RAM

vLLM — uruchomienie serwera

GPU NVIDIA (VRAM $\geq$ 24 GB), Linux, CUDA 12.x; format HF Safetensors / GGUF

Instalacja:

```
pip install vllm
python3 -c "import vllm; \
    print(vllm.__version__)"
```

Serwowanie (PLLuM-70B):

```
vllm serve \
    CYFRAGO/VPL/Llama-PLLuM-70B-chat-2512 \
    --tensor-parallel-size 2 \
    --max-model-len 8192 \
    --port 8080
```

► Pozostali vendorzy w skrócie

Po starcie serwer dostępny pod <http://localhost:8080>. Ollama: ``ollama run SpeakLeash/bielik-...`` lub ``ollama run gpt-oss:120b``; llama.cpp: ``cmake -B build -DGGML_CUDA=ON` → `llama-server --model model.gguf``.

LLM Router — AI Gateway

Open-source (Apache 2.0) warstwa pośrednicząca: aplikacja ↔ dostawca LLM

- ▶ jeden endpoint dla wielu providerów
- ▶ load balancing + health checks + streaming
- ▶ maskowanie / anonimizacja PII
- ▶ guardrails (treści zabronione)

Request → MaskerPipeline →
GuardrailPipeline →
UtilsPipeline → Model Provider

▶ **Pluginy**
fast_masker (regex, 30+ typów PII) · pii_masker (transformer) ·
nask_guard / sojka_guard · RAG, semantic routing.

Ekosystem (repozytoria):

Repo	Rola
llm-router-api	REST proxy do backendów
llm-router-lib	Python SDK (typowane)
llm-router-web	Flask UI: config + anonymizer
llm-router-plugins	maskery, guardrails, routing, RAG
llm-router-services	guardrails + PII masker (HTTP)
llm-router-utils	CLI: genai-classifier, augmentation

Automatyczne labelowanie — genai-classifier

CLI z *llm-router-utils*: JSONL bez etykiet → JSONL z polem labels

```
genai-classifier \
  --dataset-dir=resources/.../twitteremo/ \
  --prompts-dir=.../classifier \
  --output-dir=.../genailabelled \
  --llm-router-url=http://localhost:8080 \
  --model-name=gpt-oss:120b \
  --temperature=0.0 --num-workers=2 \
  --n-sample=0 --text-column-name=tekst
```

Jak działa:

buduje prompt dla każdego przykładu

wysyła do LLM przez router

parsuje wynik (klasa + confidence + reason)

batch processing, wielowątkowość, retry

♦ Koszt / czas i dobre praktyki

temperature=0.0 → determinizm i spójność etykiet · n-sample=0 → wszystkie (5k = 5k wywołań), n-sample=N do testów · num-workers ↑ = szybciej, ale większe obciążenie. Następnie konwersja do formatu treningowego (convert_genai_to_training.py).

Automatyczna augmentacja — genai-data-augmentation

CLI z *llm-router-utils*: wybór klasy → nowe warianty zachowujące etykietę

```
genai-data-augmentation \
  --dataset-path=..._labels.jsonl \
  --labels=pozytywny \
  --prompt-file=.../augmentator.prompt \
  --model-name=gpt-oss:120b \
  --n-sample=350 --n-examples=5 \
  --samples-as-examples=2 \
  --temperature=0.0
```

Przykład wyniku (1 input → 5 wariantów):

```
"tekst": "Super sprawa!",
"labels": ["pozytywny"],
"augmented": [
  "Rzeczywiście świetne!",
  "Podoba mi się!", ... ]
```

◆ Merge i strojenie

Merge: 5000 ręcznych + ~1750 (350×5) augmentowanych ≈ 6750 przykładów. `n-examples=5` to dobry balans; `samples-as-examples=2–3` (in-context); `temperature 0.0` dla spójności, `0.3–0.7` dla różnorodności (ryzyko poprawności).

Trening klasyfikatora

clarin-pl/twitteremo · 5k train / 500 valid · fine-tuning allegro/herbert-base-cased

3 warianty (postęp pipeline'u):

Wariant 1 tylko dane manualne (5k) — baseline

Wariant 2 manual + augmentacja (~1.7k, klasa pozytywny)

Wariant 3 manual + augmentacja + HIL/AL (dump z Web App)

✓ HIL/AL — wybór etykiety

Dump z Web App: brak anotacji → predykcja modelu; 1 ręczna → użyta; >1 → majority voting; remis → tie-breaker predykcją modelu.

Parametr	Wartość
Epoki	5
Batch size	32 (eff. 64, grad-accum 2)
Learning rate	3e-6
Max length	128
Scheduler	cosine
Seed	42
Logowanie	W&B (acc, F1, conf. matrix)

04

Podsumowanie i wnioski

~6 min · ocena modelu i kierunki dalsze

Co zbudowaliśmy — kompletny pipeline

- ▶ Przygotowanie datasetu (twitteremo 5k/500)
- ▶ Labelowanie LLM (genai-classifier)
- ▶ Augmentacja danych (~1.7k, klasa pozytywny)
- ▶ Merge: 5k manual + ~1.7k augment $\approx$ 6.7k
- ▶ Trening 3 wariantów (herbert-base-cased)
- ▶ Web App: ocena + anotacja (Flask + SQLite)
- ▶ HITL/AL: poprawki wracają do treningu

Ocena po HIL/AL — Weights & Biases:

- ▶ Accuracy i Macro F1 (czułe na imbalance)
- ▶ Confusion matrix — które klasy mylone
- ▶ porównanie 3 wariantów na jednym wykresie
- ▶ dane z Web App: kto i co poprawił

© Podgląd w real-time

Wyniki na żywo: dss2026.radlab.dev + Weights & Biases.

Wnioski

Co zadziało

- ▶ auto-labelowanie ~10×+ szybciej
- ▶ augmentacja ↑ recall klasy pozytywny
- ▶ Web App — szybki wgląd w błędy
- ▶ HIL/AL — najtrudniejsze przykłady

Na co uważać

- ▶ augmentacja bez HIL może szkodzić
- ▶ temperatura >0 ryzykuje poprawność
- ▶ tylko jedna klasa augmentowana
- ▶ LLM może powielać bias danych

Co dalej

- ▶ augmentacja wszystkich klas
- ▶ multi-model routing zadaniowy
- ▶ pipeline CI/CD dla datasetów
- ▶ HIL/AL na strumieniu (online)

Kontrolowana augmentacja + Active Learning + Human in the Loop = bezpieczniejsza droga do lepszego modelu.

Dziękujemy!

Pytania i dyskusja — zachęcamy do anotacji na żywo.

<https://dss2026.radlab.dev/>